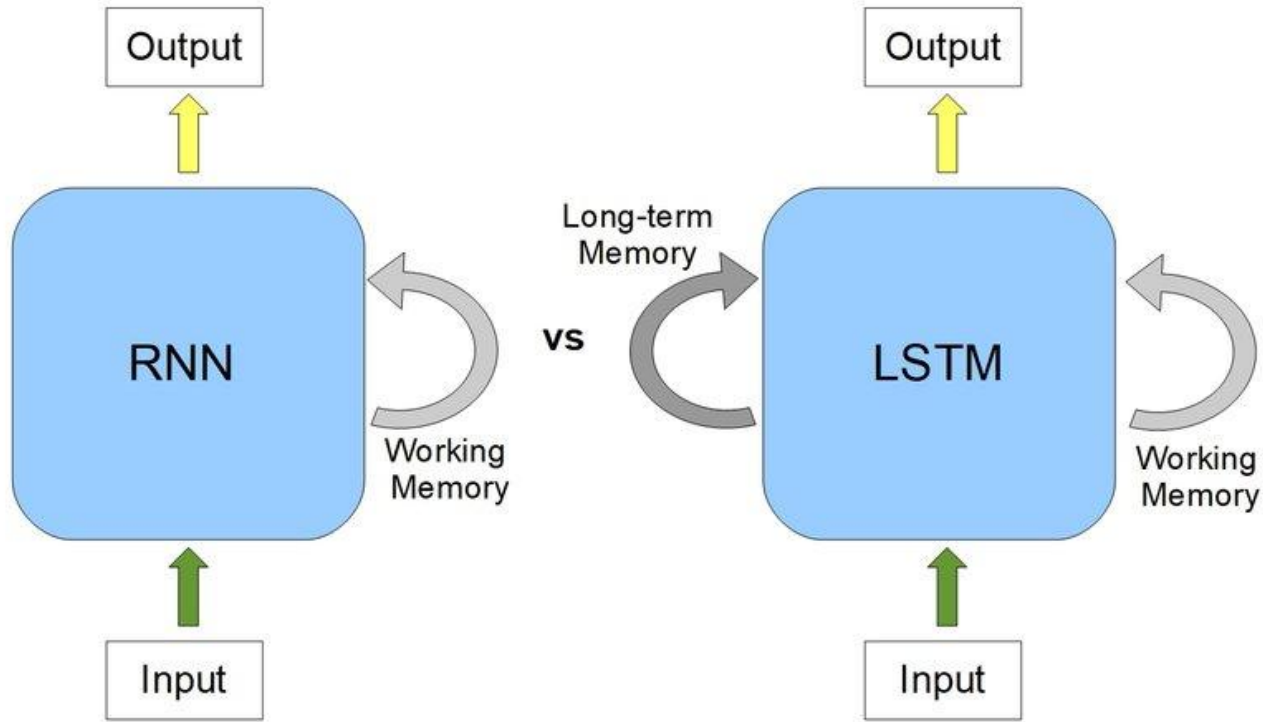


Long Short-Term Memory
(LSTM).

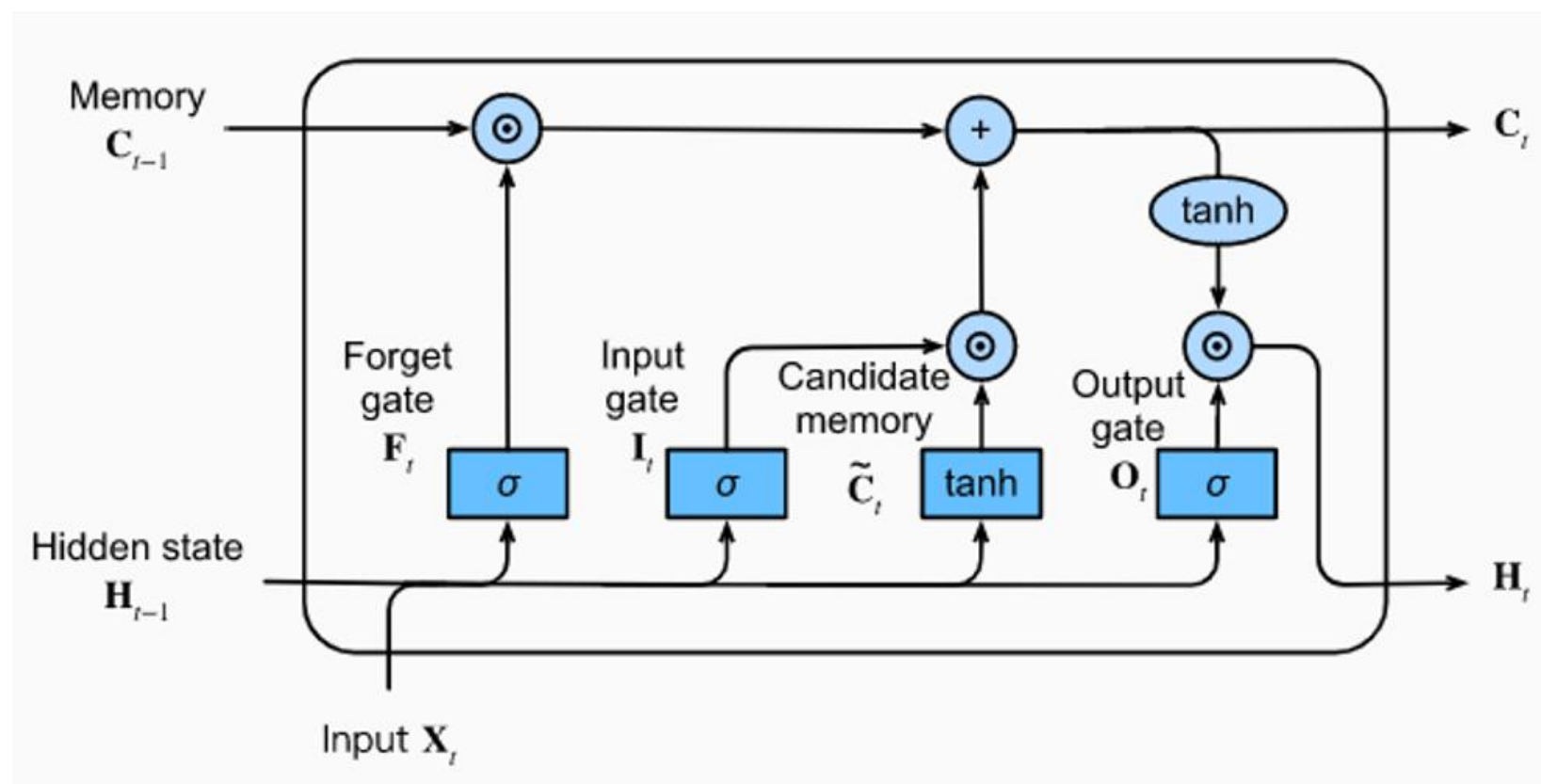


RNNs use their internal state (memory) to process sequences of inputs

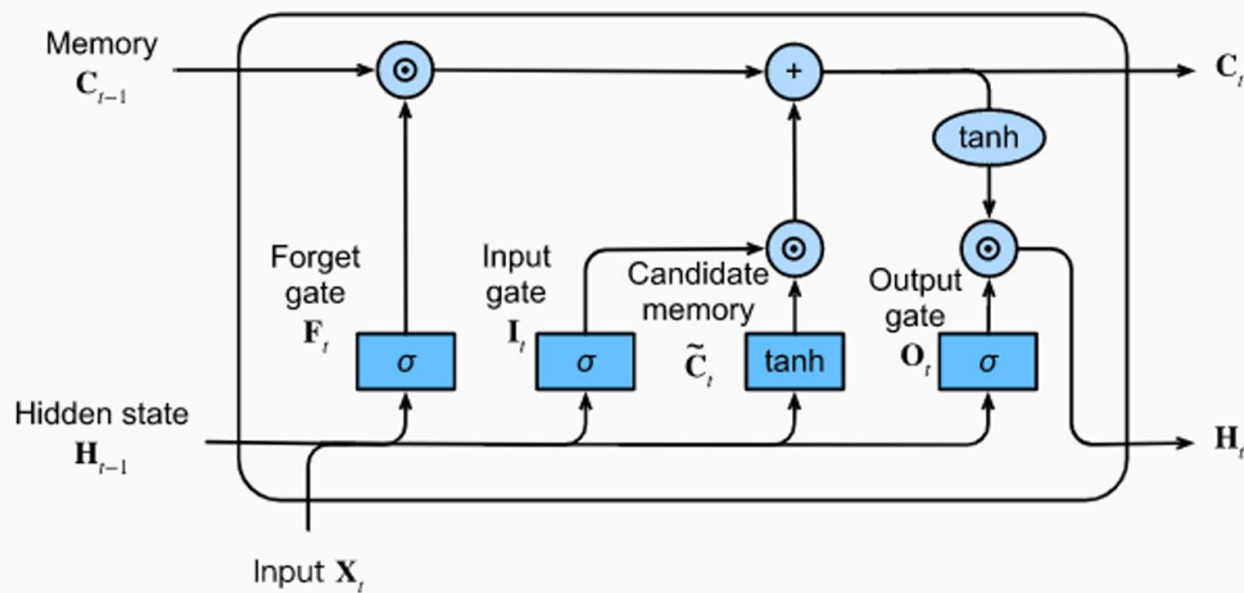
Long Short-Term Memory (LSTM) network is a variant of RNN, with additional long term memory to remember past data.

To address the **exploding gradient** and **vanishing gradient** problems, architectures like **Long Short-Term Memory (LSTM)** are designed to handle long-term dependencies.

LSTMs introduce a gating mechanism that controls how information is stored, passed, or forgotten in the network



1. **Forget Gate** (f_t) – Decides what information to forget.
2. **Input Gate** (i_t) – Decides which information to update from the input
3. **Cell State Update** ($\tilde{C}_t$) – The candidate cell state for the update.
4. **Output Gate** (o_t) – Controls what gets output from the cell.
5. **Cell State** (C_t) – Maintains the long-term memory.
6. **Hidden State** (h_t) – The current hidden state, short-term memory.



Setup (constants & weights)

Inputs / initial states / targets:

- $x_0 = 0.5, x_1 = -0.3$

- $h_0 = 0.2, c_0 = 0.1$

- $y_1 = 0.05, y_2 = 0.0$

Weights / biases (scalars, shared across timesteps):

- $W_i = 0.7, U_i = 0.10, b_i = 0.0$

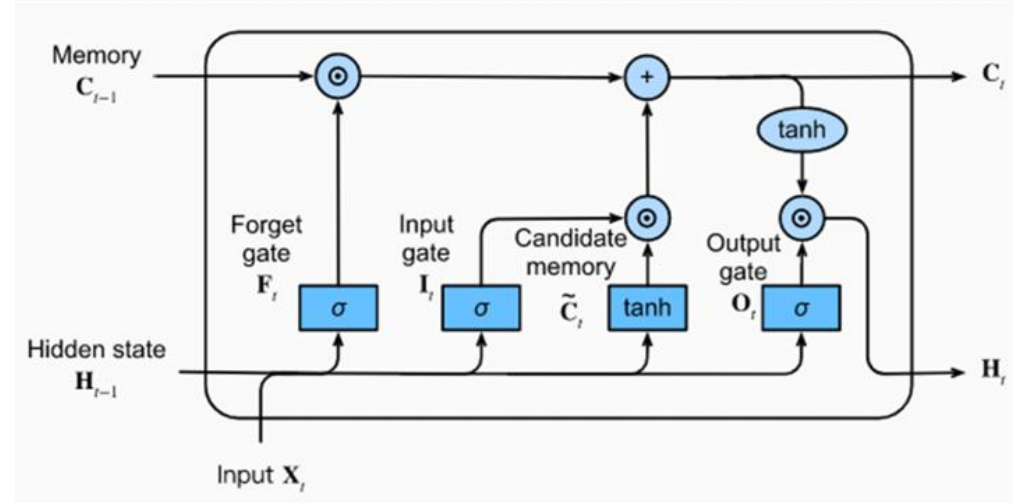
- $W_f = -0.6, U_f = 0.20, b_f = 0.10$

- $W_o = 0.3, U_o = -0.10, b_o = 0.0$

- $W_c = 0.9, U_c = 0.05, b_c = -0.05$

Activation functions: $\sigma(z) = 1/(1 + e^{-z})$.

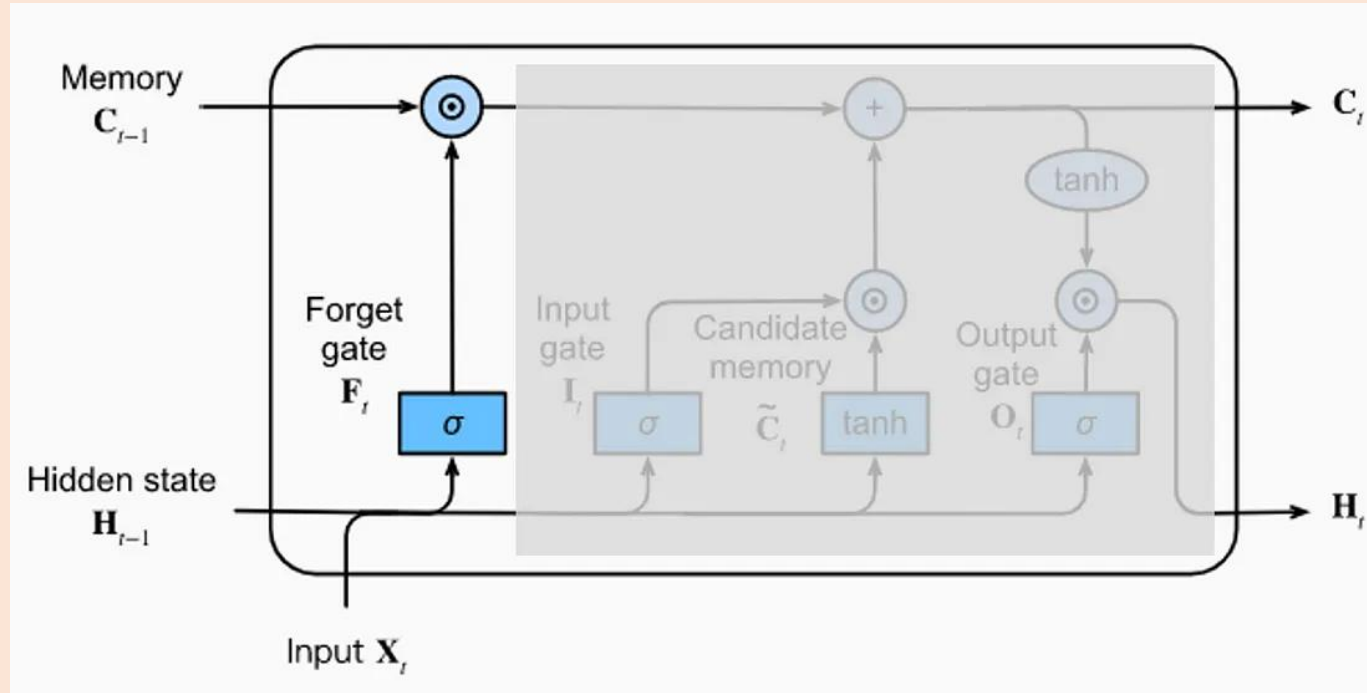
Loss per timestep: $L_t = \frac{1}{2}(h_t - y_t)^2$, total $L = L_1 + L_2$.



LSTM Forward Pass Example

The **forget gate** tells the network how much of the previous cell state to forget.

For time step $t=1$

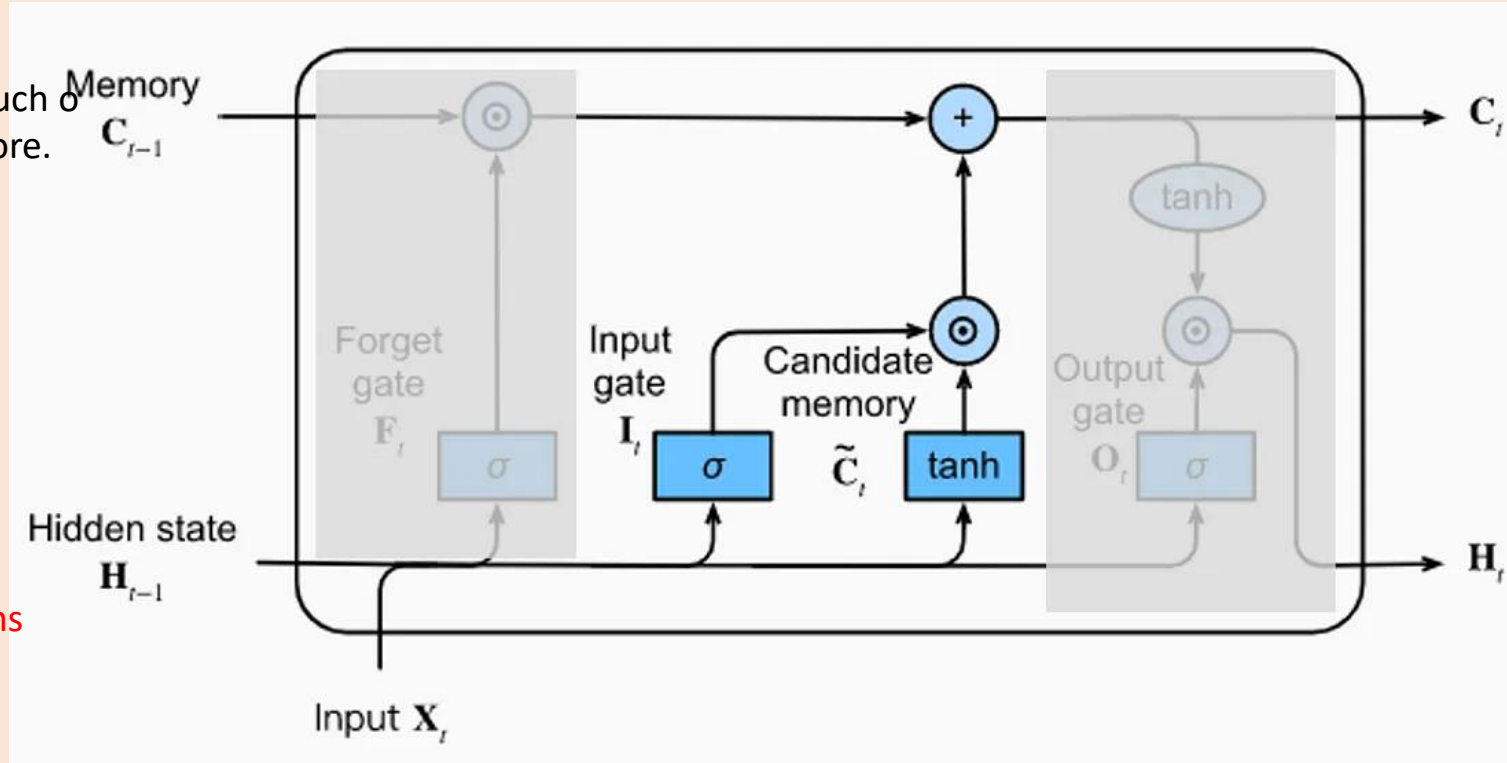


$$- 0.1 + 0.2 \cdot 0.2 + 0.5 \cdot 0.6 = 0.1600000000$$

$$f_1 = \sigma(-0.16) = 0.4600851154$$

LSTM Forward Pass Example

The **input gate** controls how much of the new candidate state to store.
For time step $t=1$



Candidate memory output neurons use hyperbolic tangent function. The properties of this function ensure that all values of the candidate vector are between -1 and 1. This is used to normalize the information that will be added to the cell state.

$$C(\tilde{C}) = 0.9 \cdot 0.5 + 0.05 \cdot 0.2 - 0.05 = 0.4100000000$$

$$= \tanh(0.41) = 0.3884726802$$

$$= +0.2 \cdot 0.1 + 0.5 \cdot 0.7 = 0.3700000000$$

$$i_1 = \sigma(0.37) = 0.5914589784$$

LSTM Forward Pass Example

The **candidate state** controls how much new information to be added

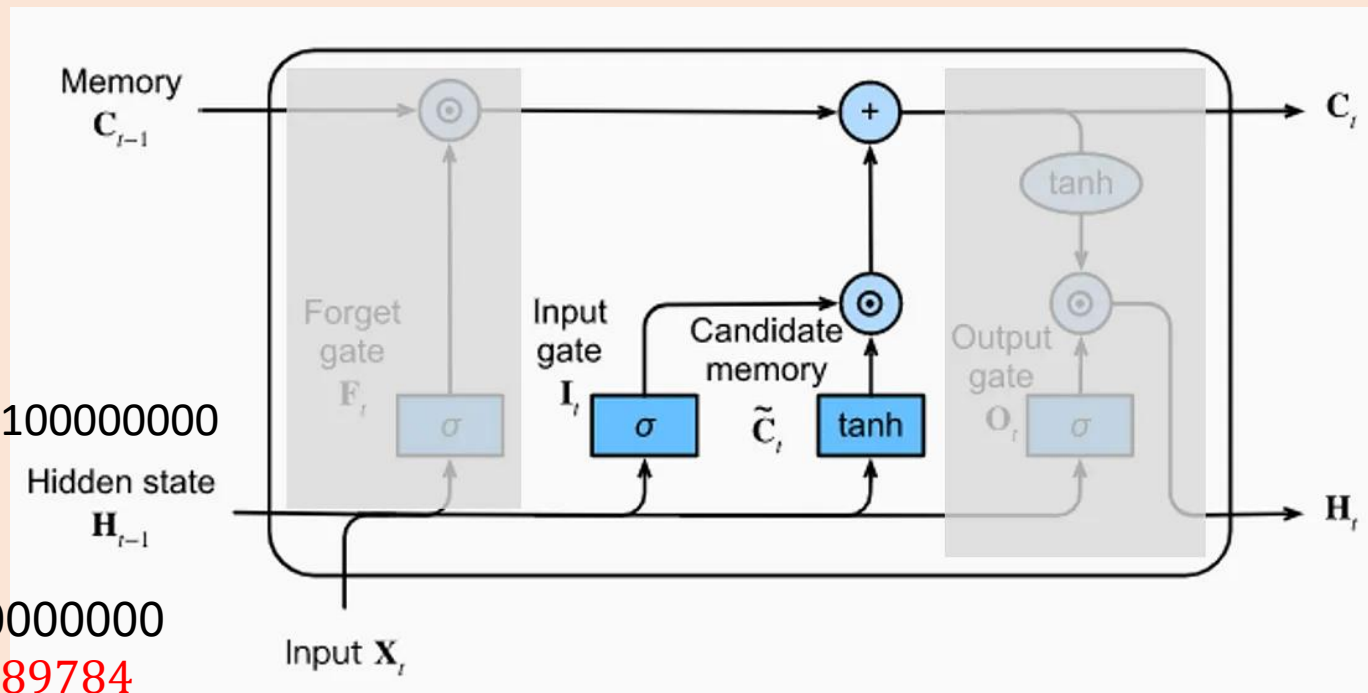
For time step $t=1$

$$C(\tilde{t}) = 0.9 \cdot 0.5 + 0.05 \cdot 0.2 - 0.05 = 0.4100000000$$

$$= \tanh(0.41) = 0.3884726802$$

$$= +0.2 \cdot 0.1 + 0.5 \cdot 0.7 = 0 + 0.3700000000$$

$$i_1 = \sigma(0.37) = 0.5914589784$$



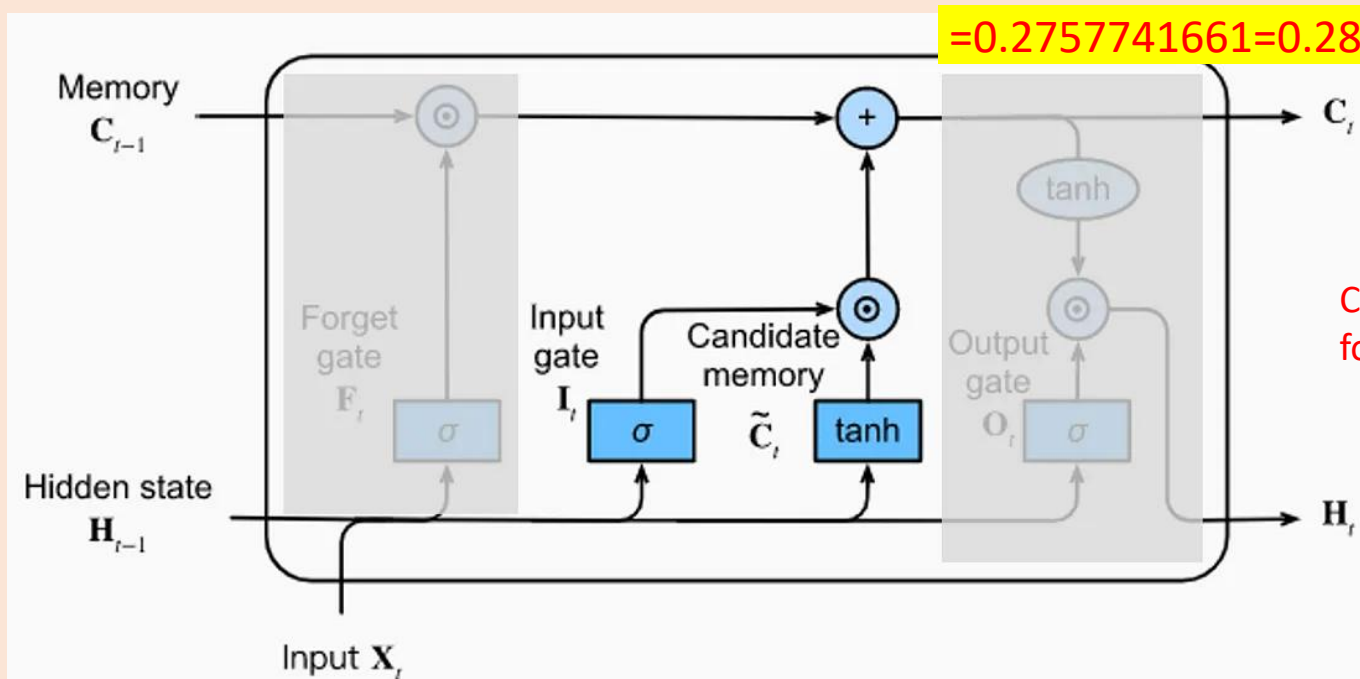
$$c_1 = f_1 c_0 + i_1 \tilde{c}_1 = 0.4600851154 \cdot 0.1 + 0.5914589784 \cdot 0.3884726802$$

$$= 0.2757741661$$

LSTM Forward Pass Example

The **cell state** controls how much new information to be added

For time step $t=1$

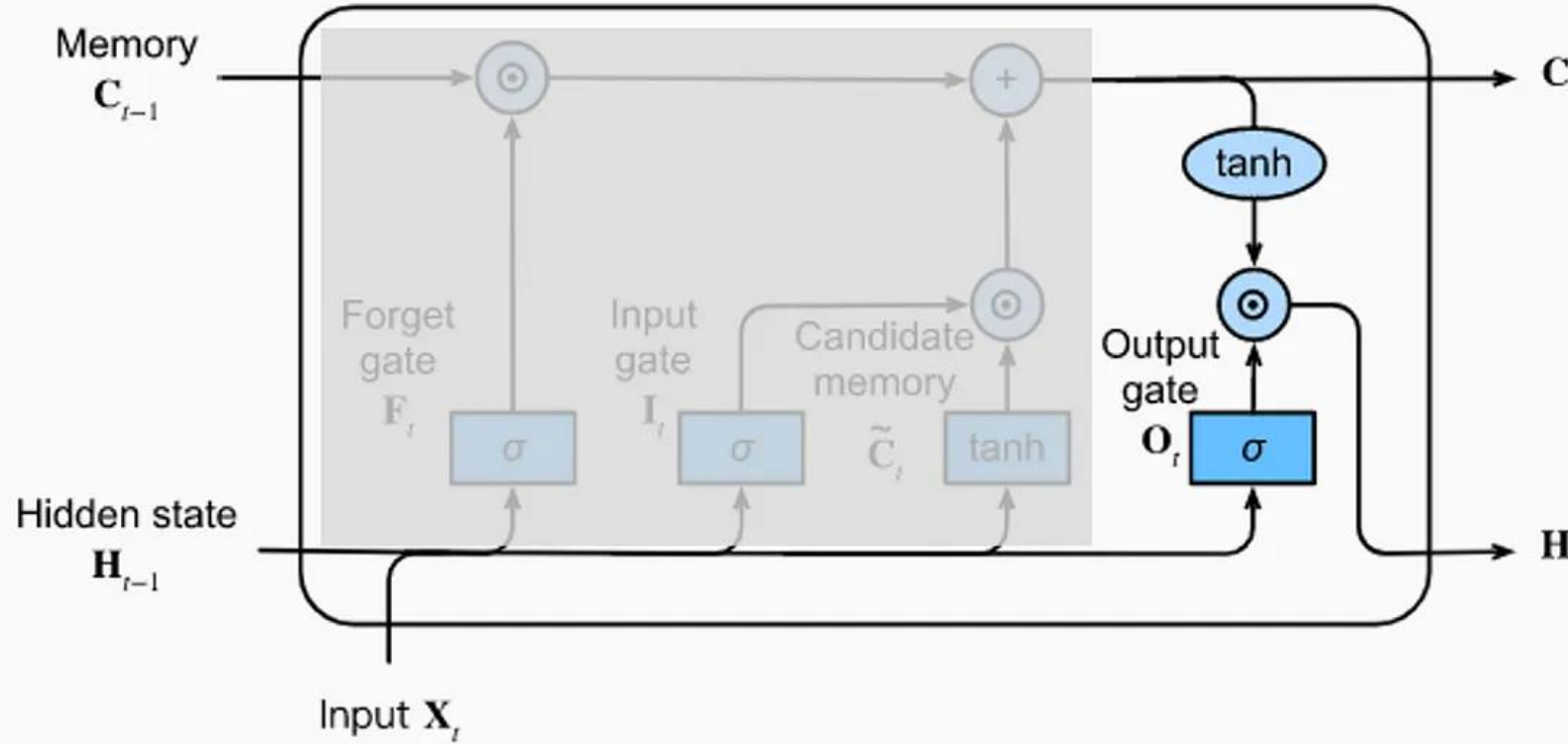


Cell state C at time t1 how much to forget + How much to add

LSTM Forward Pass Example

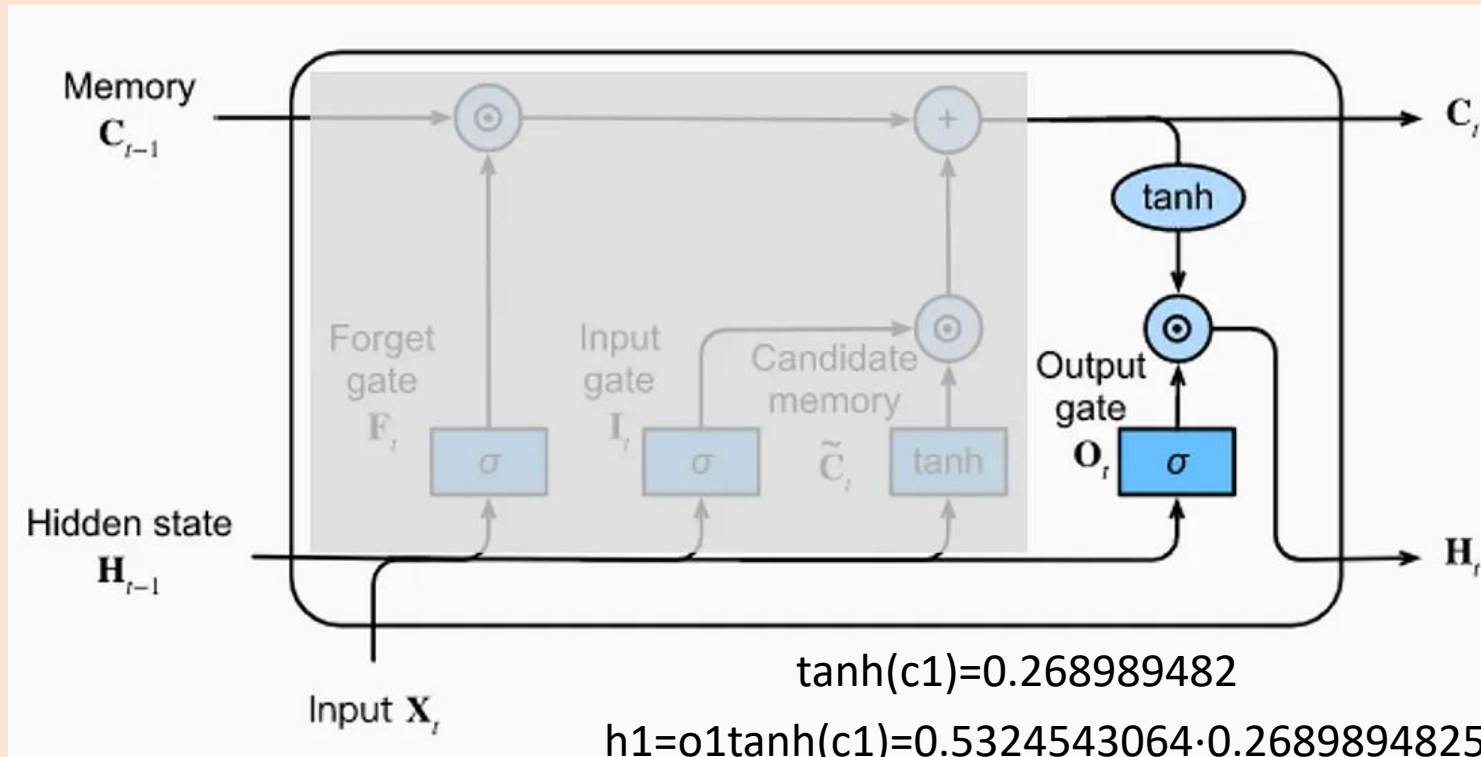
The **output gate** tells the network how much of the previous cell state to forget.

For time step $t=1$



$$z = 0.3 \cdot 0.5 - 0.1 \cdot 0.2 = 0.1300000000$$

$$o_1 = \sigma(0.13) = 0.5324543064$$

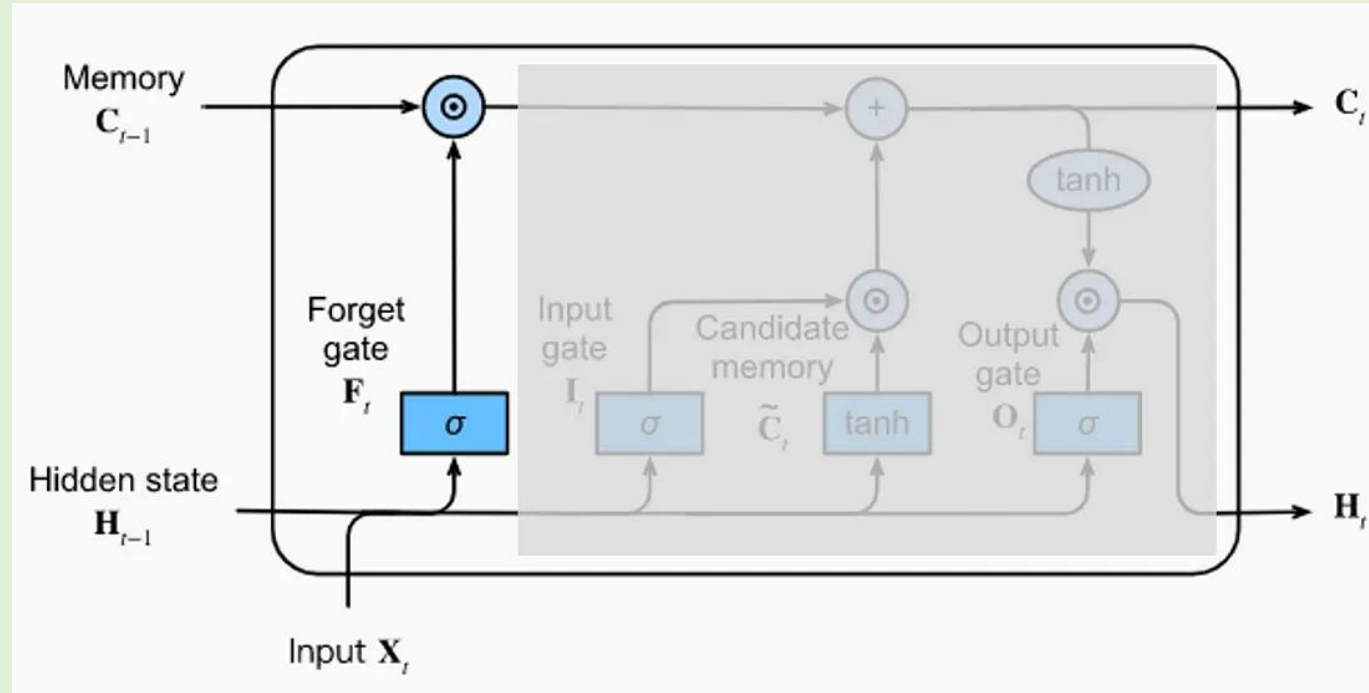


$$L1=21(h1-y1)^2=0.5\cdot(0.1432246084-0.05)^2=0.004345413801$$

LSTM Forward Pass Example

The **forget gate** tells the network how much of the previous cell state to forget.

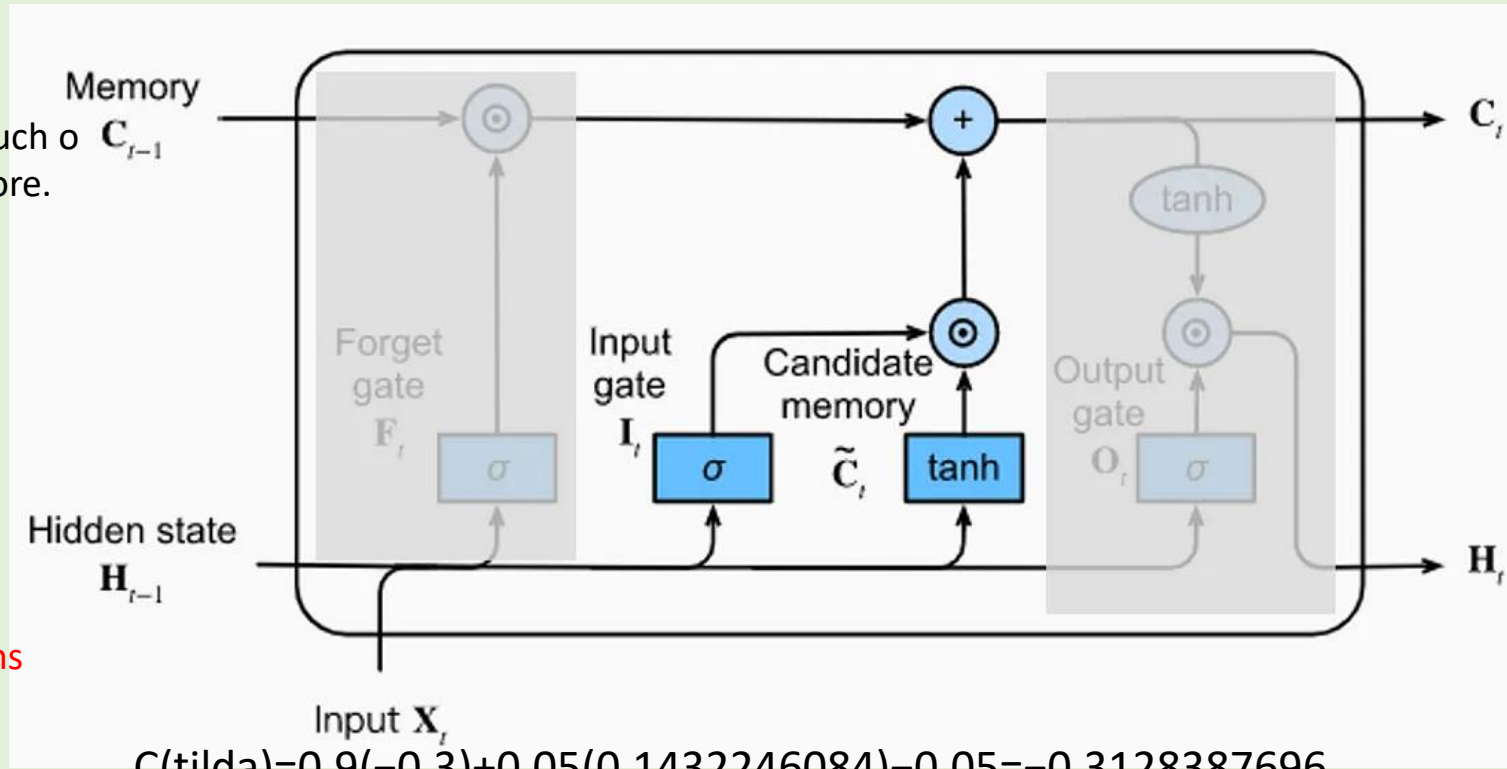
For time step $t=2$



$$0.3086449217 = 0.1 + (0.1432246084)0.2 + (0.3 -)0.6 =$$
$$f_2 = \sigma(0.3086449217) = 0.5765544675$$

LSTM Forward Pass Example

The **input gate** controls how much of the new candidate state to store.
For time step $t=1$



Candidate memory output neurons use hyperbolic tangent function. The properties of this function ensure that all values of the candidate vector are between -1 and 1. This is used to normalize the information that will be added to the cell state.

$$C(\text{tilde}) = 0.9(-0.3) + 0.05(0.1432246084) - 0.05 = -0.3128387696$$

$$\tanh(-0.3128387696) = -0.3030174248$$

$$0.1956775392 = 0 + (0.1432246084)0.1 + (0.3 -)0.7 =$$

$$i_2 = \sigma(-0.1956775392) = 0.4512361122$$

LSTM Forward Pass Example

The **candidate state** controls how much new information to be added

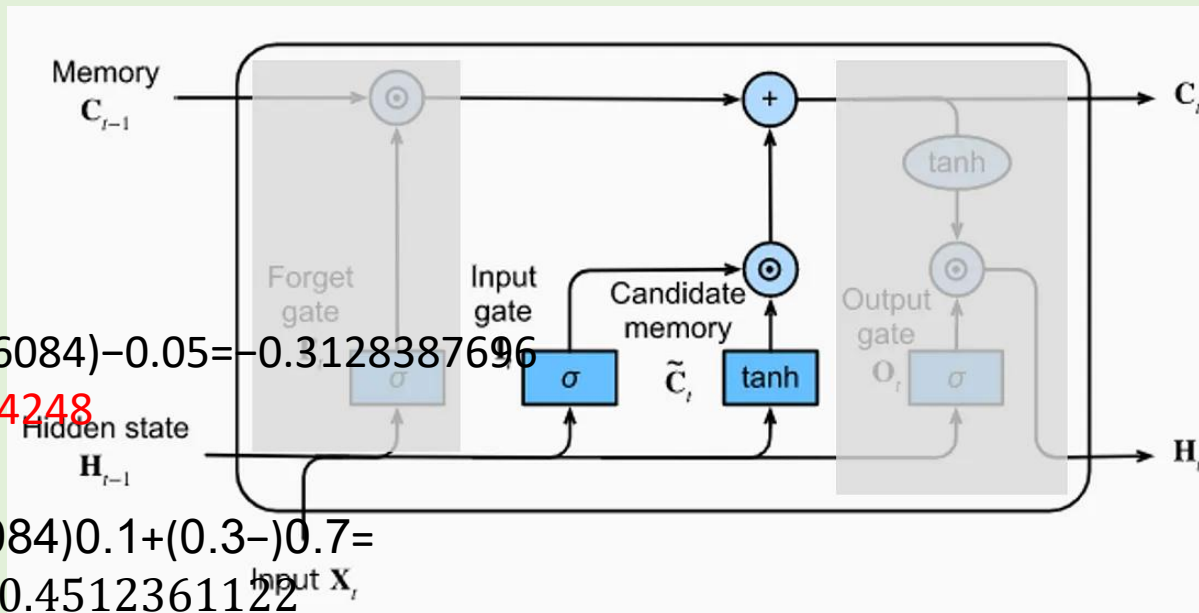
For time step $t=2$

$$C(\text{tilda}) = 0.9(-0.3) + 0.05(0.1432246084) - 0.05 = -0.3128387696$$

$$\tanh(-0.3128387696) = -0.3030174248$$

$$0.1956775392 = 0 + (0.1432246084) \cdot 0.1 + (0.3 - 0) \cdot 0.7 =$$

$$i_2 = \sigma(-0.1956775392) = 0.4512361122$$

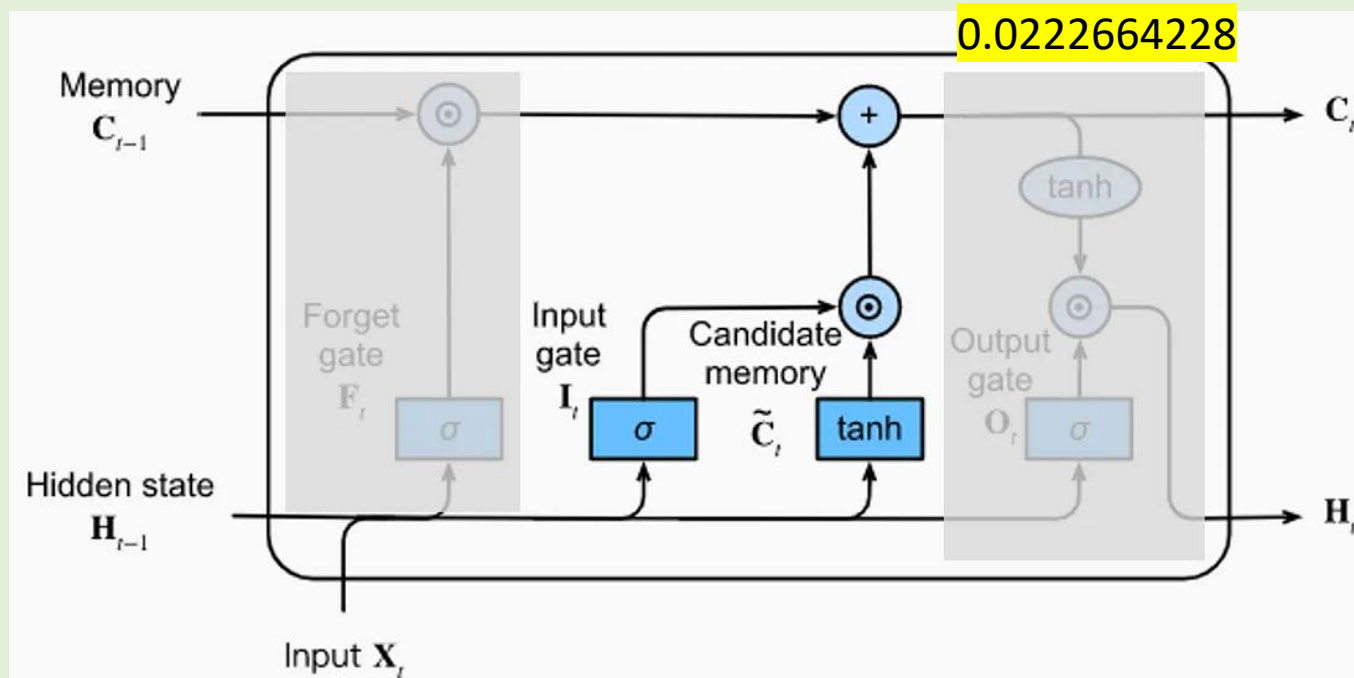


$$\begin{aligned} c_2 &= f_2 c_1 + i_2 \tilde{c}_2 = 0.5765544675 \cdot 0.2757741661 + 0.4512361122 \cdot (-0.3030174248) \\ &= 0.0222664228 \end{aligned}$$

LSTM Forward Pass Example

The **cell state** controls how much new information to be added

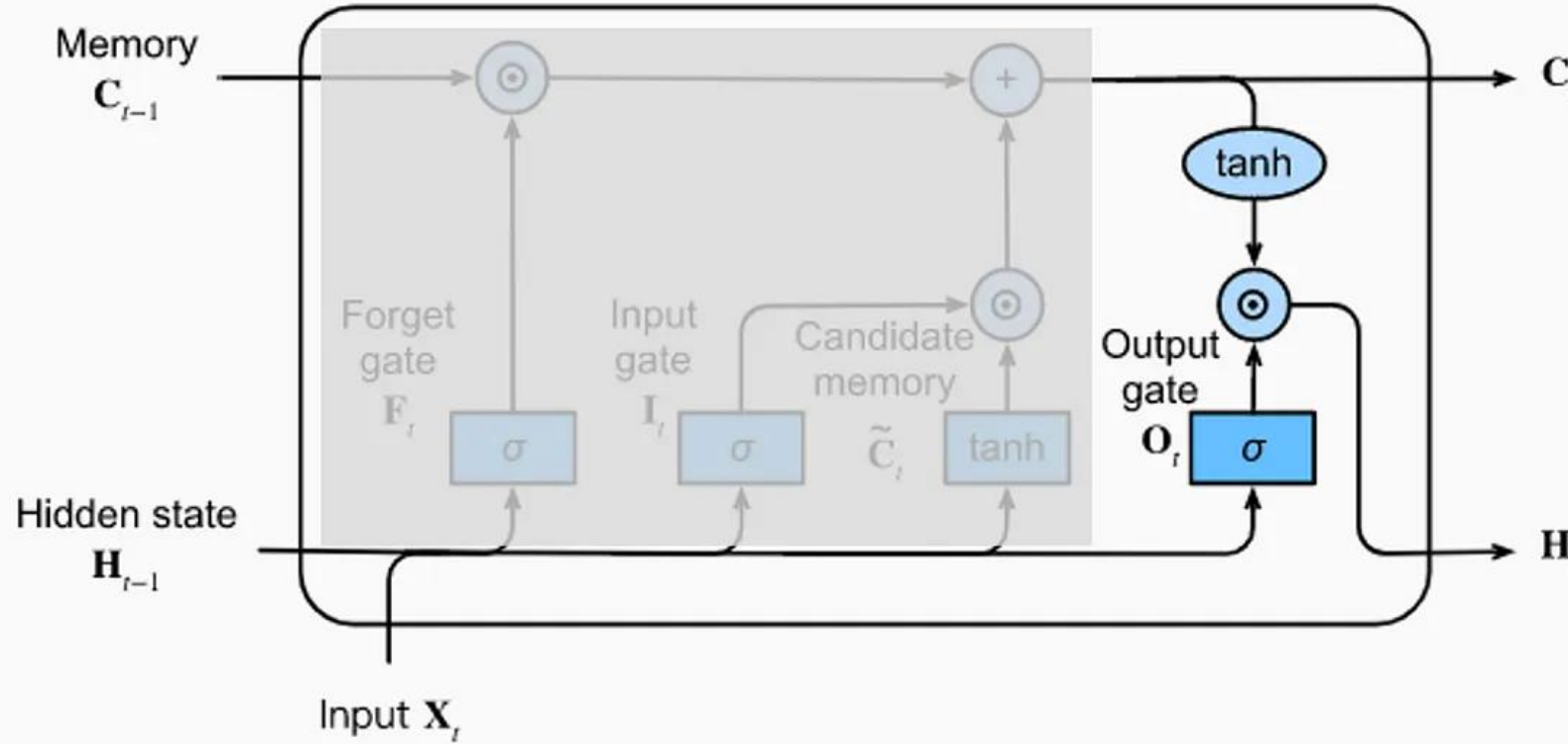
For time step $t=2$



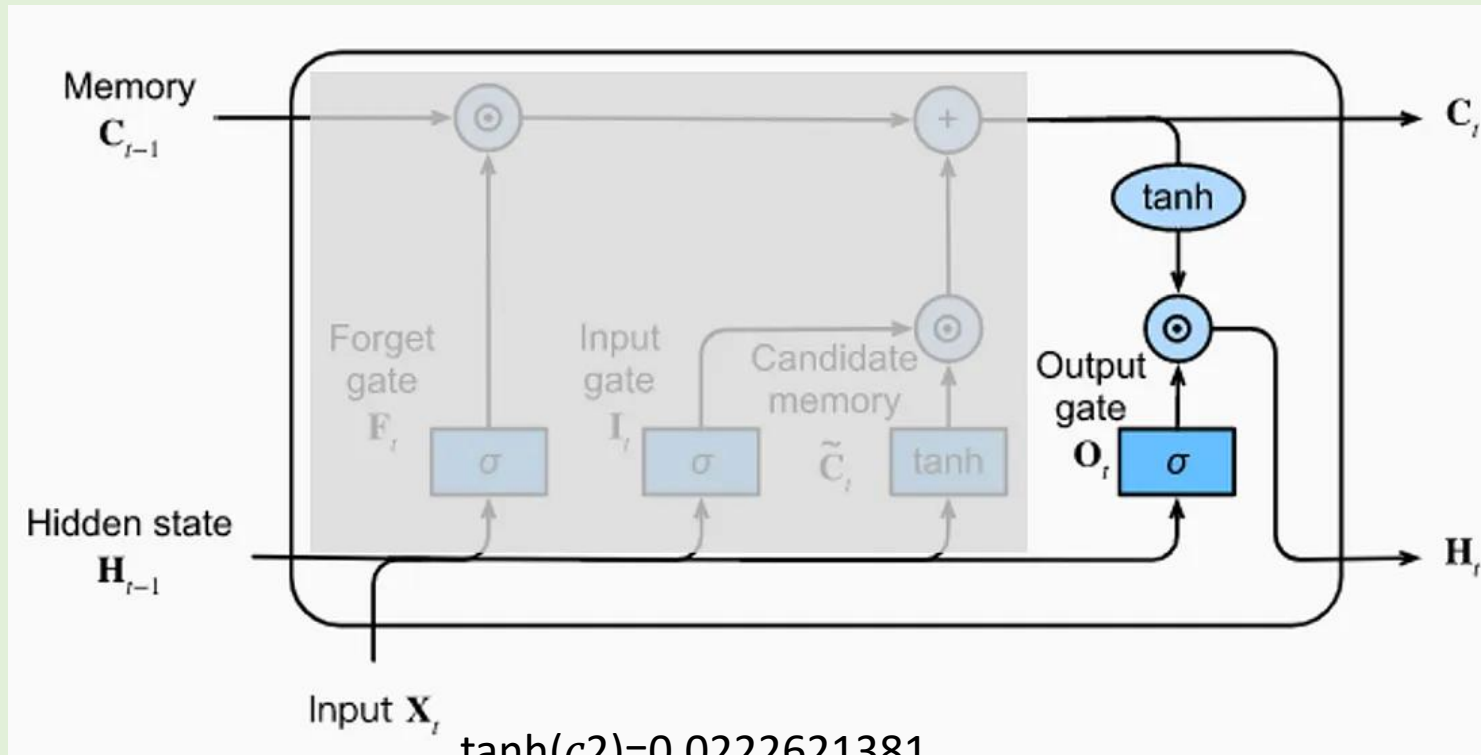
LSTM Forward Pass Example

The **output gate** tells the network how much of the previous cell state to forget.

For time step $t=2$



$$0.1043224608 = (0.1432246084)0.1 - (0.3 - 0.3) =$$
$$o_2 = \sigma(-0.1043224608) = 0.4739430124$$



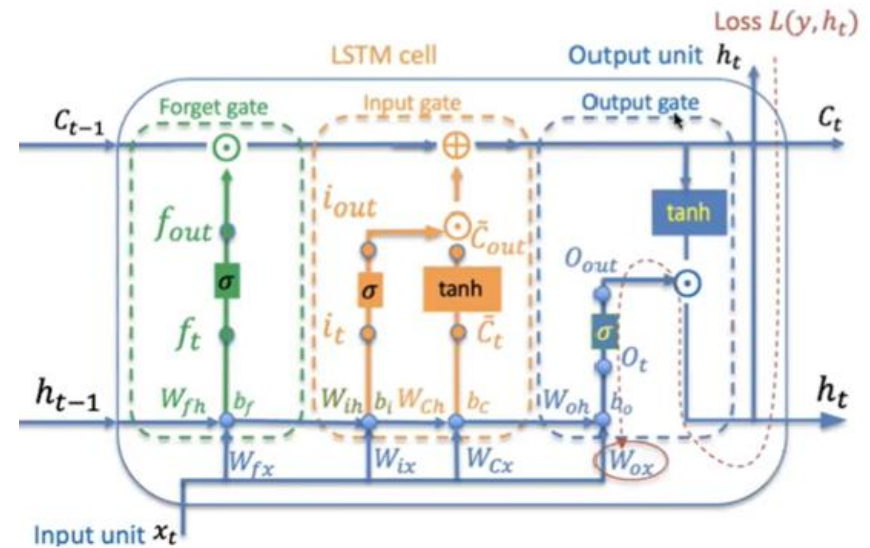
$$\tanh(c_2) = 0.0222621381$$

$$h_2 = o_2 \tanh(c_2) = 0.4739430124 \cdot 0.0222621381 = 0.0105512718$$

$$L_2 = \frac{1}{2}(h_2 - y_2)^2 = 0.5 \cdot (0.0105512718 - 0)^2 = 0.00005566466$$

$$\text{Total loss: } L = L_1 + L_2 = 0.004401078469$$

Using Backpropagation LSTM learns long-term dependencies more effectively



General rules used repeatedly

- $\frac{\partial L}{\partial h_t} = h_t - y_t$ for $L_t = \frac{1}{2}(h_t - y_t)^2$.
- For $h_t = o_t \tanh(c_t)$:

$$\frac{\partial L}{\partial o_t} = \frac{\partial L}{\partial h_t} \tanh(c_t)$$

$$\left. \frac{\partial L}{\partial c_t} \right|_{\text{from } h_t} = \frac{\partial L}{\partial h_t} o_t (1 - \tanh^2(c_t))$$

- For gates $z_t = \sigma(\text{pre}_z^t)$: $\frac{\partial L}{\partial \text{pre}_z^t} = \frac{\partial L}{\partial z_t} z_t (1 - z_t)$.
- For $g_t = \tanh(\text{pre}_g^t)$: $\frac{\partial L}{\partial \text{pre}_g^t} = \frac{\partial L}{\partial g_t} (1 - g_t^2)$.
- For cell update $c_t = f_t c_{t-1} + i_t g_t$:

$$\frac{\partial L}{\partial f_t} = \frac{\partial L}{\partial c_t} \cdot c_{t-1}, \quad \frac{\partial L}{\partial i_t} = \frac{\partial L}{\partial c_t} \cdot g_t, \quad \frac{\partial L}{\partial g_t} = \frac{\partial L}{\partial c_t} \cdot i_t$$

- **Propagation of cell gradient through time:** because c_t depends on c_{t-1} as $c_t = f_t c_{t-1} + \dots$,

$$\frac{\partial L}{\partial c_{t-1}} += f_t \cdot \frac{\partial L}{\partial c_t}.$$

Model Parameter Update

To use gradient descent method to train LSTM, loss function is defined as

$$L = -\frac{1}{2}(y - h_t)^2$$

where y is ground truth, and h_t is prediction. The parameter update equations are written as

$$W_{ox} = W_{ox} - \alpha \frac{\partial L}{\partial W_{ox}}; \quad W_{oh} = W_{oh} - \alpha \frac{\partial L}{\partial W_{oh}}; \quad b_o = b_o - \alpha \frac{\partial L}{\partial b_o} \quad (\text{output gate})$$

$$W_{fx} = W_{fx} - \alpha \frac{\partial L}{\partial W_{fx}}; \quad W_{fh} = W_{fh} - \alpha \frac{\partial L}{\partial W_{fh}}; \quad b_f = b_f - \alpha \frac{\partial L}{\partial b_f} \quad (\text{forget gate})$$

$$\left. \begin{aligned} W_{ix} &= W_{ix} - \alpha \frac{\partial L}{\partial W_{ix}}; & W_{ih} &= W_{ih} - \alpha \frac{\partial L}{\partial W_{ih}}; & b_i &= b_i - \alpha \frac{\partial L}{\partial b_i} \\ W_{cx} &= W_{cx} - \alpha \frac{\partial L}{\partial W_{cx}}; & W_{ch} &= W_{ch} - \alpha \frac{\partial L}{\partial W_{ch}}; & b_c &= b_c - \alpha \frac{\partial L}{\partial b_c} \end{aligned} \right\} (\text{input gate})$$

where α is learning factor and the gradients $\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial b}$ are computed using chain rule through the gate update equations.

LSTM Gate Update Equations

$$f_t = W_{fh}h_{t-1} + W_{fx}x_t + b_f$$

$$f_{out} = \sigma(f_t);$$

$$O_t = W_{oh}h_{t-1} + W_{ox}x_t + b_o$$

$$O_{out} = \sigma(O_t)$$

$$C_t = f_{out} \odot C_{t-1} + t_{out} \odot \tilde{C}_{out}$$

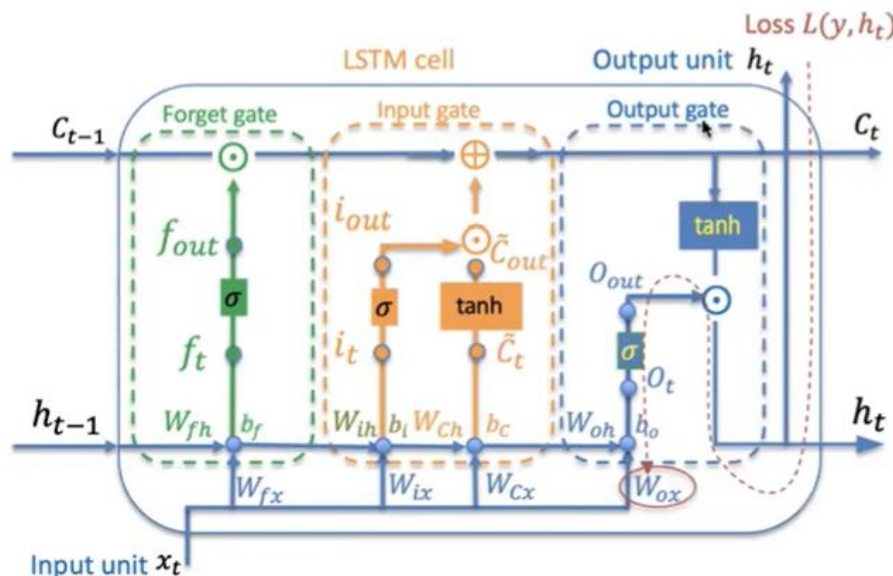
$$i_t = W_{ih}h_{t-1} + W_{ix}x_t + b_i$$

$$i_{out} = \sigma(i_t)$$

$$\tilde{C}_t = W_{Ch}h_{t-1} + W_{Cx}x_t + b_C$$

$$\tilde{C}_{out} = \tanh(\tilde{C}_t)$$

$$h_t = O_{out} \odot \tanh(C_t)$$



$$\frac{\partial L}{\partial W_{ox}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial O_{out}} \frac{\partial O_{out}}{\partial Q_t} \frac{\partial Q_t}{\partial W_{ox}} \quad (\text{Chain rule})$$

$$\frac{\partial L}{\partial h_t} = (y - h_t)$$

$$\frac{\partial h_t}{\partial O_{out}} = \tanh(C_t)$$

$$\frac{\partial O_{out}}{\partial Q_t} = O_{out}(1 - O_{out})$$

$$\frac{\partial O_t}{\partial W_{ox}} = x_t$$

$$\frac{\partial L}{\partial W_{ox}} = (y - h_t) \tanh(C_t) [O_{out}(1 - O_{out})] x_t$$

1. Gradient of Loss with Respect to Hidden State

$$dh_2 = \frac{\partial L}{\partial h_2} = h_2 - y_2 = 0.0105512718.$$

$$\frac{\partial L}{\partial W_{ox}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial O_{out}} \frac{\partial O_{out}}{\partial Q_t} \frac{\partial Q_t}{\partial W_{ox}} \quad (\text{Chain rule})$$

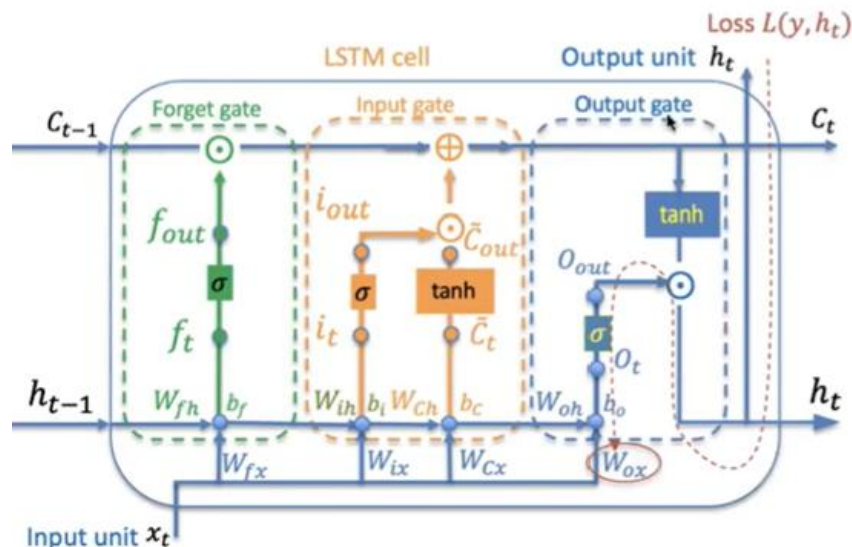
$$\frac{\partial L}{\partial h_t} = (y - h_t)$$

$$\frac{\partial h_t}{\partial O_{out}} = \tanh(C_t)$$

$$\frac{\partial O_{out}}{\partial Q_t} = O_{out}(1 - O_{out})$$

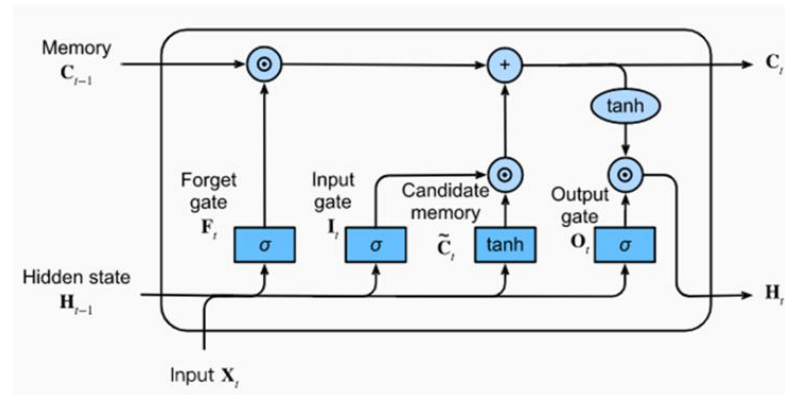
$$\frac{\partial Q_t}{\partial W_{ox}} = x_t$$

$$\frac{\partial L}{\partial W_{ox}} = (y - h_t) \tanh(C_t) [O_{out}(1 - O_{out})] x_t$$



1. Gradient of Loss with Respect to Hidden State

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial o_t} \cdot o_t \cdot (1 - o_t)$$



2. Gradient of Loss with Respect to output gate

$$\frac{\partial L}{\partial W_{ox}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial O_{out}} \frac{\partial O_{out}}{\partial Q_t} \frac{\partial Q_t}{\partial W_{ox}} \quad (\text{Chain rule})$$

$$\frac{\partial L}{\partial h_t} = (y - h_t)$$

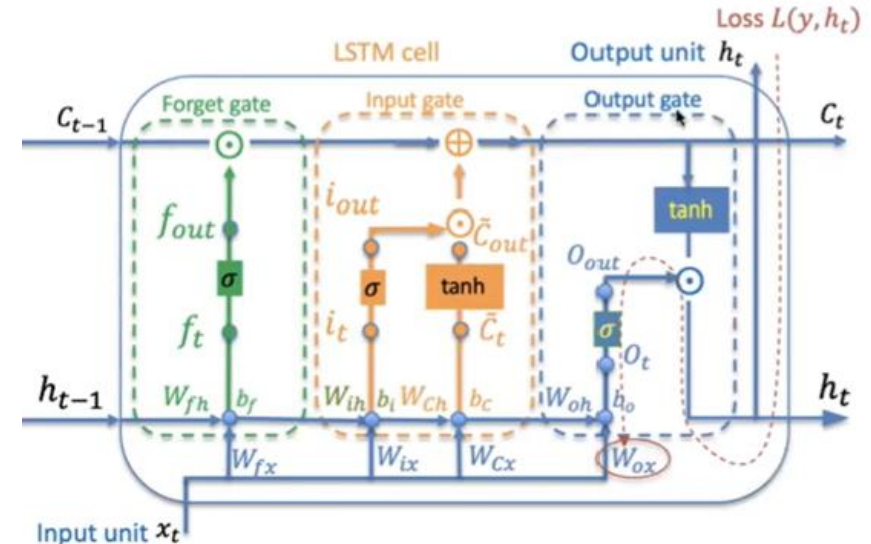
$$\frac{\partial h_t}{\partial O_{out}} = \tanh(C_t)$$

$$\frac{\partial O_{out}}{\partial Q_t} = O_{out}(1 - O_{out})$$

$$\frac{\partial Q_t}{\partial W_{ox}} = x_t$$

$$\frac{\partial L}{\partial W_{ox}} = (y - h_t) \tanh(C_t) [O_{out}(1 - O_{out})] x_t$$

$$\frac{\partial L}{\partial o_t} = \frac{\partial L}{\partial h_t} \cdot \tanh(C_t)$$



From $h_2 = o_2 \tanh(c_2)$

$$dh_2 \text{ w.r.t to } o = (dL \text{ w.r.t. to } h_2) \cdot \tanh(c_2) = 0.0105512718 \cdot 0.0222621381 = 0.0002344809.$$

$$d(zo_2) = do_2 \cdot o_2(1 - o_2) = 0.0002344809 \cdot 0.4739430124 \cdot (1 - 0.4739430124) \approx 0.000147501353.$$

$$\frac{\partial L}{\partial W_{ox}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial O_{out}} \frac{\partial O_{out}}{\partial Q_t} \frac{\partial Q_t}{\partial W_{ox}} \quad (\text{Chain rule})$$

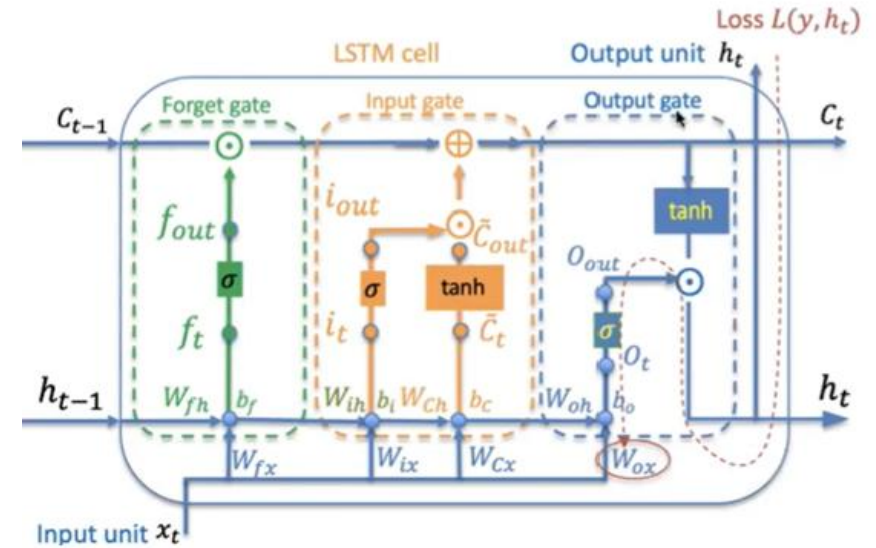
$$\frac{\partial L}{\partial h_t} = (y - h_t)$$

$$\frac{\partial h_t}{\partial O_{out}} = \tanh(C_t)$$

$$\frac{\partial O_{out}}{\partial Q_t} = O_{out}(1 - O_{out})$$

$$\frac{\partial Q_t}{\partial W_{ox}} = x_t$$

$$\frac{\partial L}{\partial W_{ox}} = (y - h_t) \tanh(C_t) [O_{out}(1 - O_{out})] x_t$$

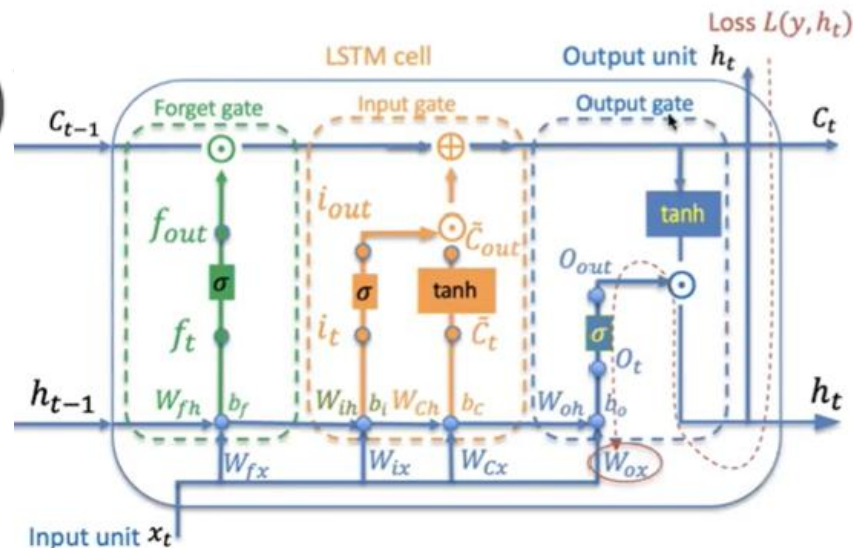


DI w.r. to $x = -0.3 * 0.000147501353$

➔ $W = \text{old weight} - \text{learning rate} * \text{dl w.r.t } w$

3. Gradient of Loss with Respect to the cell state

$$\frac{\partial L}{\partial C_t} = \frac{\partial L}{\partial h_t} \cdot o_t \cdot (1 - \tanh^2(C_t))$$



Contribution to cell from h_2 -path:

$$\begin{aligned} dc_2 &= dh_2 \cdot o_2 \cdot (1 - \tanh^2(c_2)) \\ &= 0.0105512718 \cdot 0.4739430124 \cdot (1 - 0.0222621381^2) \approx 0.004998223044. \end{aligned}$$

DI w.r.t to h , o_2

$\tanh^2$

4. Gradient of Loss with Respect to Forget Gate

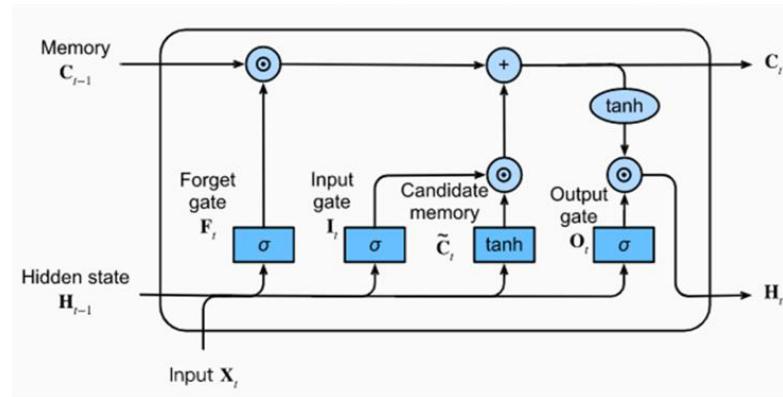
$$\frac{\partial L}{\partial f_t} = \frac{\partial L}{\partial C_t} \cdot C_{t-1}$$

Split through $c_2 = f_2 c_1 + i_2 g_2$:

$$\bullet df_2 = dc_2^{(from\ h)} \cdot c_1 \approx 0.001377708672.$$

$$d(pre_f^2) = df_2 \cdot f_2(1 - f_2) \approx 0.000470056074.$$

•.

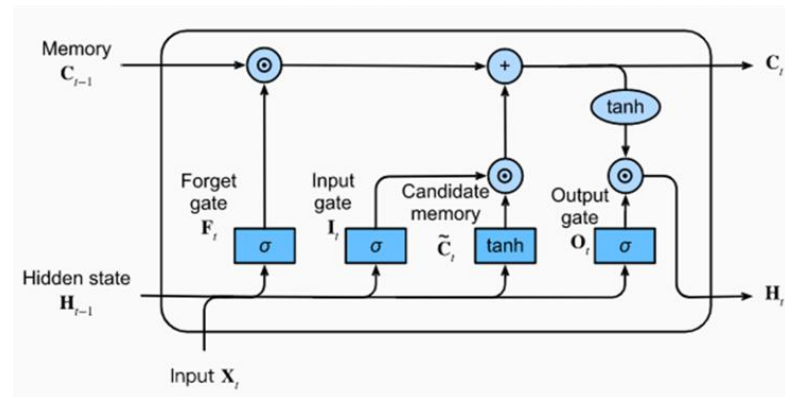


5. Gradient of Loss with Respect to Input Gate

$$\frac{\partial L}{\partial i_t} = \frac{\partial L}{\partial C_t} \cdot \tilde{C}_t$$

$$di_2 = dc_2^{(from\ h)} \cdot g_2 \approx -0.001513700888.$$

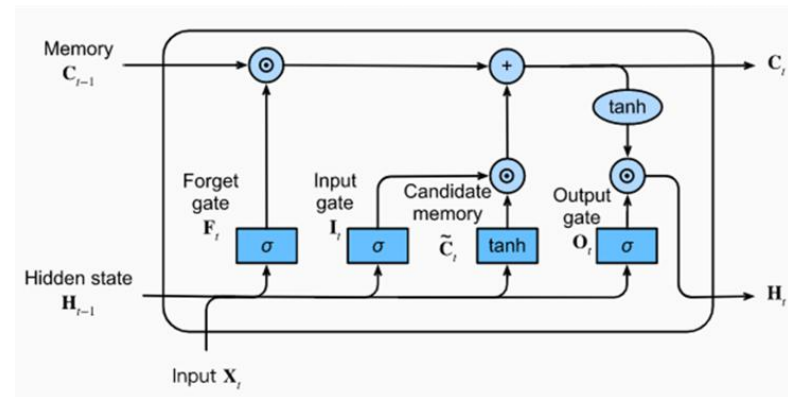
$$d(pre_i^2) = di_2 \cdot i_2(1 - i_2) \approx -0.000375540687$$



6. Gradient of Loss with Respect to Candidate cell state

$$\frac{\partial L}{\partial \tilde{C}_t} = \frac{\partial L}{\partial C_t} \cdot i_t \cdot (1 - \tilde{C}_t^2)$$

- $d\tilde{c}_2 = dc_2^{(from\ h)} \cdot i_2 \approx 0.002255954652$.
 $d(ctilda) = d\tilde{c}_2 \cdot (1 - d\tilde{c}_2^2)$
 ≈ 0.002298672329 .



7. Weight updates

Once the gradients for the gates and cell states are computed, the weight matrices W_f, W_i, W_o, W_c , and their biases are updated using gradient descent. These weights control the gates and are updated based on the gradients of their respective gates.

$$dW_i(2) = d(\text{pre}_i2) \cdot x_2 \approx -0.000375540687 \cdot (-0.3) = 0.000112510709.$$

$$dU_i(2) = d(\text{pre}_i2) \cdot h_1 \approx -0.000375540687 \cdot 0.1432246084 \approx -0.000053718322.$$

$$db_i(2) = d(\text{pre}_i2) \approx -0.000375540687$$

The cell gradient to be propagated into c_1 (from $t=2$)

$$\text{is: } dc_1(\text{from future}) = f_2 \cdot dc_2(\text{total}) = f_2 \cdot dc_2(\text{from } h) \approx 0.5765544675 \cdot 0.004998223044 = 0.002881747$$

$$826. dc_1(\text{from future}) = f_2 \cdot dc_2(\text{total}) = f_2 \cdot dc_2(\text{from } h)$$

$$\approx 0.5765544675 \cdot 0.004998223044 = 0.002881747826.$$

Gradient Computation in BPTT

Wenh

(1) Output gate: Gradients of L with respect to W_{ox} , W_{oh} and b_o

$$\frac{\partial L}{\partial W_{ox}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial O_{out}} \frac{\partial O_{out}}{\partial O_t} \frac{\partial O_t}{\partial W_{ox}} = (y - h_t) \cdot \tanh(C_t) \cdot O_{out} \cdot (1 - O_{out}) \cdot x_t$$

$$\frac{\partial L}{\partial W_{oh}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial O_{out}} \frac{\partial O_{out}}{\partial O_t} \frac{\partial O_t}{\partial W_{oh}} = (y - h_t) \cdot \tanh(C_t) \cdot O_{out} \cdot (1 - O_{out}) \cdot h_{t-1}$$

$$\frac{\partial L}{\partial b_o} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial O_{out}} \frac{\partial O_{out}}{\partial O_t} \frac{\partial O_t}{\partial b_o} = (y - h_t) \cdot \tanh(C_t) \cdot O_{out} \cdot (1 - O_{out})$$

(2) Forget gate: Gradients of L with respect to W_{fx} , W_{fh} and b_f

$$\frac{\partial L}{\partial W_{fx}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial f_{out}} \frac{\partial f_{out}}{\partial f_t} \frac{\partial f_t}{\partial W_{fx}} = (y - h_t) \cdot O_{out} \cdot (1 - \tanh^2(C_t)) \cdot C_{t-1} \cdot f_{out} \cdot (1 - f_{out}) \cdot x_t$$

$$\frac{\partial L}{\partial W_{fh}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial f_{out}} \frac{\partial f_{out}}{\partial f_t} \frac{\partial f_t}{\partial W_{fh}} = (y - h_t) \cdot O_{out} \cdot (1 - \tanh^2(C_t)) \cdot C_{t-1} \cdot f_{out} \cdot (1 - f_{out}) \cdot h_{t-1}$$

$$\frac{\partial L}{\partial b_f} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial f_{out}} \frac{\partial f_{out}}{\partial f_t} \frac{\partial f_t}{\partial b_f} = (y - h_t) \cdot O_{out} \cdot (1 - \tanh^2(C_t)) \cdot C_{t-1} \cdot f_{out} \cdot (1 - f_{out})$$

Applications

Both LSTMs and RNNs can be used for speech recognition, language modeling, and natural language processing (NLP).

LSTMs are also used for sentiment analysis and video analysis. RNNs can be used to predict daily flood levels and solve ordinal or temporal problems.